DevOps Project Report - Part 1

This document covers Phase 1, Phase 4, Phase 5 and Phase 6 of the project.

Phase 1 – Linux Server Administration

Objective

The objective of Phase 1 is to understand and implement linux server administration as part of the DevOps workflow.

Introduction

This section explains the concepts, purpose, implementation steps, and practical usage within the project. The environment used was WSL Ubuntu with Docker, Kubernetes and supporting DevOps tools. Tasks were executed using Linux terminal and validated after deployment.

Theory

This section explains the concepts, purpose, implementation steps, and practical usage within the project. The environment used was WSL Ubuntu with Docker, Kubernetes and supporting DevOps tools. Tasks were executed using Linux terminal and validated after deployment.

Implementation

This section explains the concepts, purpose, implementation steps, and practical usage within the project. The environment used was WSL Ubuntu with Docker, Kubernetes and supporting DevOps tools. Tasks were executed using Linux terminal and validated after deployment. Configuration files were created, services were deployed, verified and troubleshot until the expected result was achieved.

Workflow

This section explains the concepts, purpose, implementation steps, and practical usage within the project. The environment used was WSL Ubuntu with Docker, Kubernetes and supporting DevOps tools. Tasks were executed using Linux terminal and validated after deployment. Typical flow: Configure → Deploy → Verify → Monitor → Troubleshoot.

Best Practices

This section explains the concepts, purpose, implementation steps, and practical usage within the project. The environment used was WSL Ubuntu with Docker, Kubernetes and supporting DevOps tools. Tasks were executed using Linux terminal and validated after deployment. Use version control, least privilege, resource monitoring, descriptive naming, and validation after every change.

Outcome

This section explains the concepts, purpose, implementation steps, and practical usage within the project. The environment used was WSL Ubuntu with Docker, Kubernetes and supporting DevOps tools. Tasks were executed using Linux terminal and validated after

deployment. The phase objectives were achieved successfully and served as the foundation for later phases.

Commands Used

Example commands:
sudo apt update
docker build -t app:v1 .
kubectl apply -f deployment.yaml
helm install ...
kubectl get pods
kubectl logs <pod>
jenkins pipeline build

Phase 4 – Docker Containerization

Objective

The objective of Phase 4 is to understand and implement docker containerization as part of the DevOps workflow.

Introduction

This section explains the concepts, purpose, implementation steps, and practical usage within the project. The environment used was WSL Ubuntu with Docker, Kubernetes and supporting DevOps tools. Tasks were executed using Linux terminal and validated after deployment.

Theory

This section explains the concepts, purpose, implementation steps, and practical usage within the project. The environment used was WSL Ubuntu with Docker, Kubernetes and supporting DevOps tools. Tasks were executed using Linux terminal and validated after deployment.

Implementation

This section explains the concepts, purpose, implementation steps, and practical usage within the project. The environment used was WSL Ubuntu with Docker, Kubernetes and supporting DevOps tools. Tasks were executed using Linux terminal and validated after deployment. Configuration files were created, services were deployed, verified and troubleshot until the expected result was achieved.

Workflow

This section explains the concepts, purpose, implementation steps, and practical usage within the project. The environment used was WSL Ubuntu with Docker, Kubernetes and supporting DevOps tools. Tasks were executed using Linux terminal and validated after deployment. Typical flow: Configure → Deploy → Verify → Monitor → Troubleshoot.

Best Practices

This section explains the concepts, purpose, implementation steps, and practical usage within the project. The environment used was WSL Ubuntu with Docker, Kubernetes and supporting DevOps tools. Tasks were executed using Linux terminal and validated after deployment. Use version control, least privilege, resource monitoring, descriptive naming, and validation after every change.

Outcome

This section explains the concepts, purpose, implementation steps, and practical usage within the project. The environment used was WSL Ubuntu with Docker, Kubernetes and supporting DevOps tools. Tasks were executed using Linux terminal and validated after deployment. The phase objectives were achieved successfully and served as the foundation for later phases.

Commands Used

Example commands:
sudo apt update
docker build -t app:v1 .
kubectl apply -f deployment.yaml
helm install ...
kubectl get pods
kubectl logs <pod>
jenkins pipeline build

Phase 5 – Kubernetes Deployment

Objective

The objective of Phase 5 is to understand and implement kubernetes deployment as part of the DevOps workflow.

Introduction

This section explains the concepts, purpose, implementation steps, and practical usage within the project. The environment used was WSL Ubuntu with Docker, Kubernetes and supporting DevOps tools. Tasks were executed using Linux terminal and validated after deployment.

Theory

This section explains the concepts, purpose, implementation steps, and practical usage within the project. The environment used was WSL Ubuntu with Docker, Kubernetes and supporting DevOps tools. Tasks were executed using Linux terminal and validated after deployment.

Implementation

This section explains the concepts, purpose, implementation steps, and practical usage within the project. The environment used was WSL Ubuntu with Docker, Kubernetes and

supporting DevOps tools. Tasks were executed using Linux terminal and validated after deployment. Configuration files were created, services were deployed, verified and troubleshot until the expected result was achieved.

Workflow

This section explains the concepts, purpose, implementation steps, and practical usage within the project. The environment used was WSL Ubuntu with Docker, Kubernetes and supporting DevOps tools. Tasks were executed using Linux terminal and validated after deployment. Typical flow: Configure → Deploy → Verify → Monitor → Troubleshoot.

Best Practices

This section explains the concepts, purpose, implementation steps, and practical usage within the project. The environment used was WSL Ubuntu with Docker, Kubernetes and supporting DevOps tools. Tasks were executed using Linux terminal and validated after deployment. Use version control, least privilege, resource monitoring, descriptive naming, and validation after every change.

Outcome

This section explains the concepts, purpose, implementation steps, and practical usage within the project. The environment used was WSL Ubuntu with Docker, Kubernetes and supporting DevOps tools. Tasks were executed using Linux terminal and validated after deployment. The phase objectives were achieved successfully and served as the foundation for later phases.

Commands Used

Example commands:
sudo apt update
docker build -t app:v1 .
kubectl apply -f deployment.yaml
helm install ...
kubectl get pods
kubectl logs <pod>
jenkins pipeline build

Phase 6 – CI/CD Pipeline (Jenkins)

Objective

The objective of Phase 6 is to understand and implement ci/cd pipeline (jenkins) as part of the DevOps workflow.

Introduction

This section explains the concepts, purpose, implementation steps, and practical usage within the project. The environment used was WSL Ubuntu with Docker, Kubernetes and supporting DevOps tools. Tasks were executed using Linux terminal and validated after deployment.

Theory

This section explains the concepts, purpose, implementation steps, and practical usage within the project. The environment used was WSL Ubuntu with Docker, Kubernetes and supporting DevOps tools. Tasks were executed using Linux terminal and validated after deployment.

Implementation

This section explains the concepts, purpose, implementation steps, and practical usage within the project. The environment used was WSL Ubuntu with Docker, Kubernetes and supporting DevOps tools. Tasks were executed using Linux terminal and validated after deployment. Configuration files were created, services were deployed, verified and troubleshot until the expected result was achieved.

Workflow

This section explains the concepts, purpose, implementation steps, and practical usage within the project. The environment used was WSL Ubuntu with Docker, Kubernetes and supporting DevOps tools. Tasks were executed using Linux terminal and validated after deployment. Typical flow: Configure → Deploy → Verify → Monitor → Troubleshoot.

Best Practices

This section explains the concepts, purpose, implementation steps, and practical usage within the project. The environment used was WSL Ubuntu with Docker, Kubernetes and supporting DevOps tools. Tasks were executed using Linux terminal and validated after deployment. Use version control, least privilege, resource monitoring, descriptive naming, and validation after every change.

Outcome

This section explains the concepts, purpose, implementation steps, and practical usage within the project. The environment used was WSL Ubuntu with Docker, Kubernetes and supporting DevOps tools. Tasks were executed using Linux terminal and validated after deployment. The phase objectives were achieved successfully and served as the foundation for later phases.

Commands Used

Example commands:
sudo apt update
docker build -t app:v1 .
kubectl apply -f deployment.yaml
helm install ...
kubectl get pods
kubectl logs <pod>
jenkins pipeline build

DevOps Project Report - Part 2

This document covers Phase 7, Phase 8, Phase 9 and Phase 10 of the project.

Phase 7 – Monitoring (Prometheus & Grafana)

Objective

Objective for Monitoring (Prometheus & Grafana). This phase focuses on implementing and understanding the technology as part of the DevOps lifecycle.

Introduction

Introduction for Monitoring (Prometheus & Grafana). The tools were deployed on Kubernetes running in WSL and validated after configuration.

Theory

Theory for Monitoring (Prometheus & Grafana). Core concepts, architecture, and purpose are explained before implementation.

Implementation

Implementation for Monitoring (Prometheus & Grafana). Resources were deployed, configured, verified, and tested using Kubernetes and Helm where applicable.

Workflow

Workflow for Monitoring (Prometheus & Grafana). Deploy → Configure → Verify → Troubleshoot → Validate.

Best Practices

Best Practices for Monitoring (Prometheus & Grafana). Use least privilege, monitor continuously, validate changes, and document configurations.

Outcome

Outcome for Monitoring (Prometheus & Grafana). The objectives of the phase were completed and integrated with the remaining DevOps workflow.

Commands Used

helm install ...
kubectl get pods -A
kubectl logs <pod>
kubectl describe pod <pod>
kubectl port-forward ...
curl http://service
kubectl exec -it <pod> -- sh

Phase 8 – Centralized Logging (ELK Stack)

Objective

Objective for Centralized Logging (ELK Stack). This phase focuses on implementing and understanding the technology as part of the DevOps lifecycle.

Introduction

Introduction for Centralized Logging (ELK Stack). The tools were deployed on Kubernetes running in WSL and validated after configuration.

Theory

Theory for Centralized Logging (ELK Stack). Core concepts, architecture, and purpose are explained before implementation.

Implementation

Implementation for Centralized Logging (ELK Stack). Resources were deployed, configured, verified, and tested using Kubernetes and Helm where applicable.

Workflow

Workflow for Centralized Logging (ELK Stack). Deploy → Configure → Verify → Troubleshoot → Validate.

Best Practices

Best Practices for Centralized Logging (ELK Stack). Use least privilege, monitor continuously, validate changes, and document configurations.

Outcome

Outcome for Centralized Logging (ELK Stack). The objectives of the phase were completed and integrated with the remaining DevOps workflow.

Commands Used

helm install ...
kubectl get pods -A
kubectl logs <pod>
kubectl describe pod <pod>
kubectl port-forward ...
curl http://service
kubectl exec -it <pod> -- sh

Phase 9 – Security Hardening

Objective

Objective for Security Hardening. This phase focuses on implementing and understanding the technology as part of the DevOps lifecycle.

Introduction
Introduction for Security Hardening. The tools were deployed on Kubernetes running in WSL and validated after configuration.

Theory
Theory for Security Hardening. Core concepts, architecture, and purpose are explained before implementation.

Implementation
Implementation for Security Hardening. Resources were deployed, configured, verified, and tested using Kubernetes and Helm where applicable.

Workflow
Workflow for Security Hardening. Deploy → Configure → Verify → Troubleshoot → Validate.

Best Practices
Best Practices for Security Hardening. Use least privilege, monitor continuously, validate changes, and document configurations.

Outcome
Outcome for Security Hardening. The objectives of the phase were completed and integrated with the remaining DevOps workflow.

Commands Used
helm install ...
kubectl get pods -A
kubectl logs <pod>
kubectl describe pod <pod>
kubectl port-forward ...
curl http://service
kubectl exec -it <pod> -- sh

Phase 10 – Backup & Disaster Recovery

Objective
Objective for Backup & Disaster Recovery. This phase focuses on implementing and understanding the technology as part of the DevOps lifecycle.

Introduction
Introduction for Backup & Disaster Recovery. The tools were deployed on Kubernetes running in WSL and validated after configuration.

Theory
Theory for Backup & Disaster Recovery. Core concepts, architecture, and purpose are explained before implementation.

Implementation

Implementation for Backup & Disaster Recovery. Resources were deployed, configured, verified, and tested using Kubernetes and Helm where applicable.

Workflow

Workflow for Backup & Disaster Recovery. Deploy → Configure → Verify → Troubleshoot → Validate.

Best Practices

Best Practices for Backup & Disaster Recovery. Use least privilege, monitor continuously, validate changes, and document configurations.

Outcome

Outcome for Backup & Disaster Recovery. The objectives of the phase were completed and integrated with the remaining DevOps workflow.

Commands Used

helm install ...
kubectl get pods -A
kubectl logs <pod>
kubectl describe pod <pod>
kubectl port-forward ...
curl http://service
kubectl exec -it <pod> -- sh